

RETO 2 — MAPEO GENERAL

María Alexandra Jiménez Suarez
Juan José Álvarez Restrepo
Carolina Ramírez Lotero
Natalia Giraldo Morales

Universidad Pontificia Bolivariana, Facultad de Ingeniería
Arquitectura de Software

Cesar Augusto López Gallego

30 de agosto de 2026

MAPEO GENERAL

1. Estructura general del proyecto antiguo

Capa	Contenido
Clases	Producto (abstracta), Medicamento, MedicamentoCapsula, MedicamentoLiquido, Comestible, Cosmetico, Persona, Cliente, Usuario, Laboratorio, Movimiento
Interfaces	IRepositorio*, INotificador*, IDescuento, IServicioNotificacion
Repositorios	Implementaciones por archivo de texto (uno por tipo de producto + cliente + usuario)
Servicios	ServicioProducto, ServicioCliente, ServicioUsuario, ServicioMovimiento, ServicioAutenticacion, ServicioDescuento, ServicioNotificacion
Eventos	EventoStockMinimo, EventoVencimiento, EventoPuntos, EventoMovimiento (implementan las interfaces de notificación)
Factories(no se tocó)	ProductoFactory (solo cápsulas y líquidos con valores hardcodeados)
Aspectos	AspectoAutenticacion, AspectoValidacion (clases estáticas)
App	Program.cs actúa como composition root + UI de consola

2. Revisión SOLID

Principio	Estado actual	Punto de dolor
S – Single Responsibility	Servicios mezclan orquestación + persistencia en memoria + notificaciones. Program.cs hace composition, login, menú y lógica de negocio.	Cambiar la forma de mostrar alertas o la UI obliga a tocar el núcleo.
O – Open/Closed	Cerrado para extensión de productos: hay que crear nueva clase + nuevo repositorio + modificar carga en Program.	Agregar un tipo de producto es caro.
L – Liskov	Se cumple razonablemente (subtipos de Producto se usan vía la base).	Bajo riesgo actual.
I – Interface Segregation	Interfaces de notificación muy finas (bien). Interfaces de repositorio también separadas.	Buen punto.

D – Dependency Inversion	Servicios dependen de abstracciones (IRepositorio*, INotificador*). Composition root en Program instancia todo con new.	No hay contenedor DI; el composition root es frágil y difícil de testear.
--------------------------	--	---

3. Responsabilidades

Componente	Responsabilidad actual	Problema
ServicioProducto	Lista en memoria + verificación stock/vencimiento + carga desde archivo	Mezcla dominio y persistencia
ServicioCliente	Lista en memoria + acumulación de puntos + carga	Idempotencia
ProductoFactory	Solo crea 2 tipos con valores fijos (stockMin=5, fechas fijas, Gel, Vidrio...)	No es una factory real de dominio; es un helper incompleto
AspectoValidacion / AspectoAutenticacion	Validación y login estáticos	No son aspectos (AOP); son utilidades estáticas difíciles de inyectar/mockear
Program.cs	Todo: DI manual, suscripción a eventos, menú, reglas de venta	Concentra muchos datos

4. Creación de objetos

Lugar	Cómo se crean	Problema
RepositorioMedicamentoArchivo	new MedicamentoCapsula(...) + new Laboratorio(...) + hardcodeado	Toda la lógica de construcción está en el repositorio
RepositorioComestibleArchivo / Cosmetico	new directo del tipo concreto	Idempotencia
ProductoFactory	Solo 2 métodos con valores mágicos	Casi no se usa; la carga real no pasa por ella
Program	new de todos los servicios, eventos y repos	Composition root manual, sin DI container

Composición

- Composición buena: Medicamento compone Laboratorio; Movimiento compone Producto.

Composición ausente / débil:

- No hay un agregado "Farmacia" o "Inventario" que agrupe productos, movimientos y clientes.
- Los servicios mantienen sus propias List<T> en memoria, estado duplicado y desincronizado potencial.
- Notificaciones se hacen por eventos, pero la suscripción está hardcodeada en Program (no hay bus de eventos ni mediador).

Condicionales

No hay un switch o cadena de if por tipo de producto en el dominio (positivo), pero la discriminación por tipo se resolvió con repositorios separados, lo que genera duplicación.

Ubicación	Condicional	Riesgo
ServicioProducto.VerificarStock	if (Stock <= StockMinimo)	Bajo
ServicioProducto.VerificarVencimiento	if (dias <= 30)	Umbral mágico
AspectoValidacion	Validaciones de nombre/cédula/precio/stock	Dispersas; no hay reglas de dominio centralizadas
Program (menú)	switch (opcion) grande	Todo el flujo de UI y parte de lógica de negocio
Carga de productos	Tres llamadas distintas a CargarDesdeArchivo según tipo	Condicional implícito por tipo de repositorio

5. Mapa de dependencias

Componente	Depende de	Tipo de dependencia	¿Concreta o abstracción?	Observación
ServicioProducto	IRepositorioProducto	Acceso a productos	Abstracción	Correcto

ServicioProducto	INotificadorStockMínimo	Notificación	Abstracción	Correcto
ServicioProducto	INotificadorVencimiento	Notificación	Abstracción	Correcto
ServicioCliente	IRepositorioCliente	Persistencia	Abstracción	Correcto
ServicioCliente	INotificadorPuntos	Notificación	Abstracción	Correcto
ServicioUsuario	IRepositorioUsuario	Persistencia	Abstracción	Correcto
ServicioMovimiento	INotificadorMovimiento	Notificación	Abstracción	Correcto
ServicioAutenticacion	List<Usuario>	Datos de usuarios	Concreta	Dependencia a revisar
Program	implementaciones concretas	Composición	Concreta	Punto rígido
RepositorioMedicamentoArchivo	MedicamentoCapsula	Creación	Concreta	Punto rígido
RepositorioCosmeticoArchivo	Cosmetico	Creación	Concreta	Punto rígido
RepositorioComestibleArchivo	Comestible	Creación	Concreta	Punto rígido

- **Correcto**

Una dependencia es Correcta cuando la relación entre dos elementos está bien justificada por el diseño actual y no genera un problema relevante de evolución.

- **Dependencia a revisar**

Aquí hay algo que no necesariamente está mal, pero queremos analizarlo porque podría convertirse en problema dependiendo de cómo evolucione el sistema.

- **Punto rígido**

Un punto rígido es una dependencia o estructura donde existe evidencia de que: un cambio futuro obliga a modificar código existente, aumenta el número de lugares que hay que tocar o concentra una decisión que puede crecer.